

Programming Basics 1



08: Vererbung und Komposition

Sommersemester 2024

Simon Dewert

HAW Hamburg

Fakultät Design, Medien und Information

E52, Finkenau 35, 22081 Hamburg

simon.dewert@haw-hamburg.de

Übersicht dieser Lehreinheit

1. Nachtrag
 - a. Compilation Unit
 - b. Kapselung
2. Vererbung: Prinzip
3. Vererbung in Java
4. Verdeckung von Attributen
5. Konstruktoren abgeleiteter Klassen
6. Überschreiben von Methoden
7. Überladen von Methoden
8. Vererbung vs. Komposition

Nachtrag: Compilation Unit

Eine Quellcode-Datei wird auch **Compilation Unit** genannt und darf auch **mehr als eine** Top-Level Klasse enthalten:

```
public class Programm
{
    public static void main(String[] args)
    {
        ZweiteKlasse meinObjekt = new ZweiteKlasse();
    }
}

class ZweiteKlasse
{
}
```

Bedingungen: Eine der Klassen muss den Namen der Quellcode-Datei haben und nur diese Klasse **darf public** sein!

Nachtrag: Kapselung

Die Attribute einer Klasse werden meistens durch den Zugriffsmodifizierer **private** vor direktem Zugriff von außen geschützt. Stattdessen geschieht der Zugriff indirekt, z. B. über öffentliche sogenannte "**Getter**"- und "**Setter**"-Methoden:

```
private int meinAttribut = 0;
```

```
public void setMeinAttribut(int wert)
{
    meinAttribut = wert;
}
```

„**Setter**“-Methode des Attributs **meinAttribut**

```
public int getMeinAttribut()
{
    return meinAttribut;
}
```

„**Getter**“-Methode des Attributs **meinAttribut**

Eine derartige Zugriffskontrolle wird **Kapselung** genannt.

Aufgabe 8.1



Fügen der Klasse **Pokemon** **Getter-** und **Setter** für **alle private Attribute** hinzu.

Beispiel der vorherigen Folie:

```
private int meinAttribut = 0;

public void setMeinAttribut(int wert)
{
    meinAttribut = wert;
}

public int getMeinAttribut()
{
    return meinAttribut;
}
```

Pokémon
<pre>-name: String -lvl: int -exp: int +typ: Type -health: int -maxHealth: int -attacks: List<Attack> +myTrainer: Trainer</pre>
<pre>+doDamage(Pokemon p): void +getName(): String +getHealth(): int ~takeDamage(int dmg): void +toString(): String +setLevel(int lvl): void +getLevel(): int +getExp(): int +getAttacks(): List<Attack> +addExp(int value): void -checkLevelUp(): void +addAttack(Attack atk): void</pre>

Prinzip der Vererbung

Bei der Vererbung wird eine neue Klasse von einer bestehenden Klasse (der sog. Basisklasse) im Sinn einer Hierarchie "**abgeleitet**".

Dadurch "**erbt**" die abgeleitete Klasse **alle Elemente** (Attribute und Methoden) ihrer Basisklasse bis auf die **Konstruktoren**.

Im **UML 2.0 Klassendiagramm** wird für diesen Zusammenhang zwischen zwei Klassen ein spezieller **Pfeiltyp** verwendet:



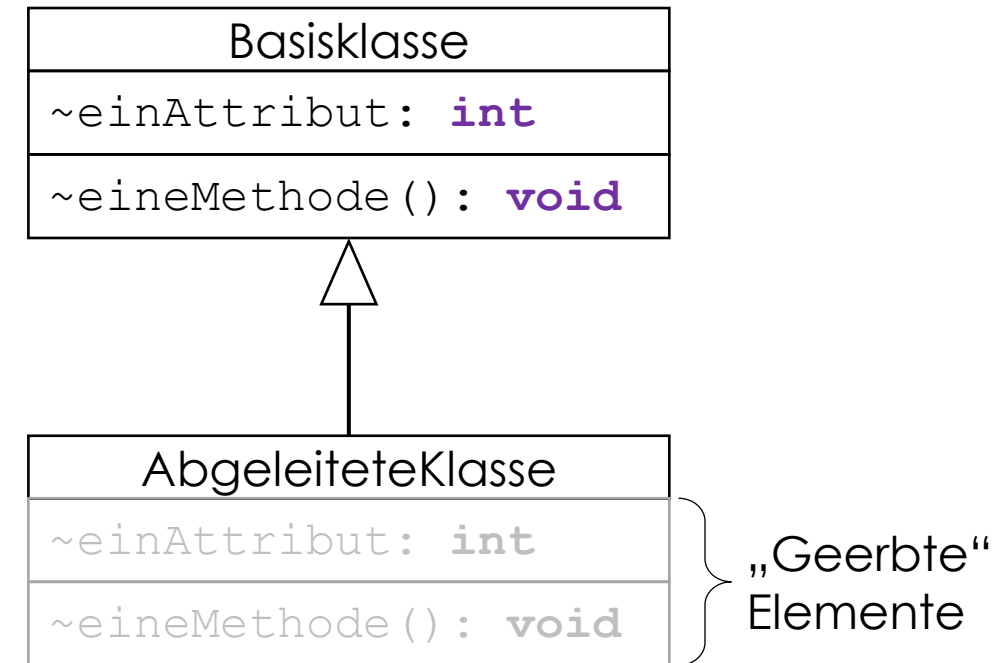
Vererbung in Java

Eine Klasse wird in Java von einer anderen Klasse abgeleitet, indem ihrer Deklaration das Schlüsselwort **extends** mit nachfolgender Angabe der Basisklasse folgt:

```
public class BasisKlasse
{
    int einAttribut = 0;

    void eineMethode()
    {
        [...]
    }
}

class AbgeleiteteKlasse extends BasisKlasse
{
}
```



Vererbung in Java: Wichtige Regeln

- Es werden immer alle Elemente außer den Konstruktoren vererbt. Ein selektiver Ausschluss ist nicht möglich.
- Vererbte Elemente behalten ihren jeweiligen **Zugriffsschutz**; als **private** deklarierte Elemente werden ebenfalls vererbt, sind aber nicht direkt zugreifbar.
- Eine Klasse kann nur von **höchstens einer** anderen Klasse abgeleitet werden; darüber hinaus haben alle Java-Klassen eine gemeinsame oberste Basisklasse: *java.lang.Object*
- Von einer mit dem Schlüsselwort **final** versehenen Klasse kann **nicht** abgeleitet werden.

Der Zugriffsmodifizierer **protected**

Wenn ein Klassenelement (Attribut oder Methode) mit dem Zugriffsmodifizierer **protected** versehen wurde, so können alle Klassen desselben Pakets sowie **alle abgeleiteten Klassen direkt** auf dieses Element zugreifen.

Zugriffs- modifizierer	Zugriff von innerhalb....			
	...der selben Klasse	...von Klassen desselben Pakets	...von Unterklassen in beliebigen Paketen	...von Klassen in beliebigen Paketen
public	Ja	Ja	Ja	Ja
protected	Ja	Ja	Ja	Nein
(keine)	Ja	Ja	Nein	Nein
private	Ja	Nein	Nein	Nein

Konstruktor abgeleiteter Klassen 1/2

Abgeleitete Klassen können **eigene Konstruktoren** besitzen, mit deren Hilfe ihren **klassenspezifischen Attributen** individuelle Anfangswerte gegeben werden.

Im Konstruktor wird immer erst ein Konstruktor der Basisklasse aufgerufen. Falls nicht anders angegeben der Default-Konstruktor der Basisklasse.

```
BasisKlasse();
```

```
class AbgeleiteteKlasse extends BasisKlasse {  
  
    private int attribut = 0;  
  
    AbgeleiteteKlasse(int attribut) {  
        this.attribut = attribut; //Wertzuweisung des eigenen Attributs  
    }  
}
```

Konstruktor abgeleiteter Klassen 2/2

Geerbte Attribute erhalten ihre Anfangswerte jedoch vom **Konstruktor der Basisklasse**. Dieser wird im Konstruktor der abgeleiteten Klasse mit **super(...)** explizit **als erstes Statement** aufgerufen:

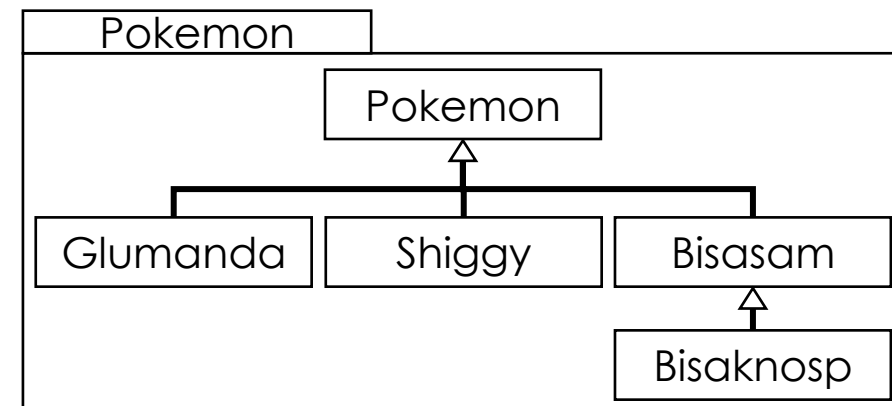
```
class AbgeleiteteKlasse extends BasisKlasse {  
  
    private int attribut = 0;  
  
    AbgeleiteteKlasse(int basisAttribut, int attribut) {  
        super(basisAttribut); // Aufruf des Basisklassen-Konstruktors  
        this.attribut = attribut; // Wertzuweisung des eigenen Attributs  
    }  
}
```

Ist kein **super()**-Aufruf vorhanden, wird implizit der Standard-Konstruktor der Basisklasse aufgerufen!

Aufgabe 8.2



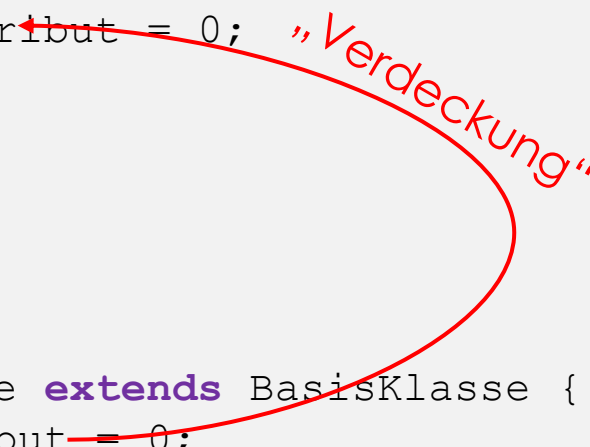
- a) Ändern Sie die Zugriffsrechte des **Konstruktors** der Klasse **Pokemon** auf **protected**
- b) Erstellen Sie **public** Konstruktoren, ohne Übergabeparameter, für alle vier Pokémon (Glumanda, Shiggy, Bisasam & Bisaknosp). Die Basiswerte für das Pokemon werden im Konstruktor festlegen.
Hinweis: Bisasam wird mit Aufgabe c einen zweiten Konstruktor benötigen.
- c) Bilden Sie die Vererbung dieser Klassen nach folgendem UML-Diagramm ab.



Verdeckung von Attributen 1/2

Wenn eine abgeleitete Klasse ein bereits in der Basisklasse definiertes **Attribut erneut definiert**, so "**verdeckt**" dieses **klassenspezifische Attribut** das Attribut der Basisklasse:

```
public class BasisKlasse {  
    protected int einAttribut = 0;  
  
    void eineMethode() {  
        einAttribut = 10;  
    }  
}  
  
class AbgeleiteteKlasse extends BasisKlasse {  
    private int einAttribut = 0;  
}
```



Achtung:

Die geerbte Methode **eineMethode()** arbeitet weiterhin mit dem (verdeckten) Attribut **einAttribut** der Basisklasse

Verdeckung von Attributen 2/2

Auf verdeckte Attribute kann innerhalb der abgeleiteten Klasse über das Schlüsselwort **super** zugegriffen werden:

```
public class BasisKlasse {  
    protected int einAttribut = 0;  
}  
  
class AbgeleiteteKlasse extends BasisKlasse {  
    private int einAttribut = 0;  
  
    void eineMethode() {  
        this.einAttribut = super.einAttribut;  
    }  
}
```

this.

Zugriff auf das eigene Attribut
einAttribut

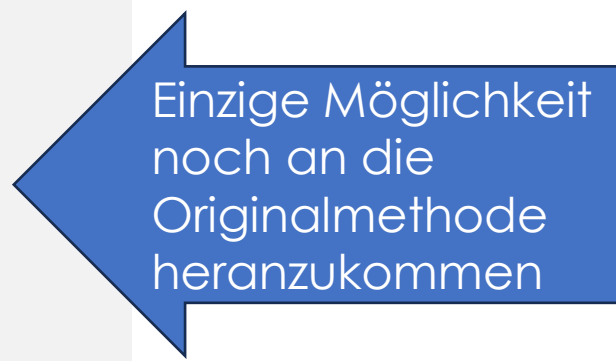
super.

Zugriff auf das verdeckte Attribut
einAttribut der Basisklasse

Überschreiben von Methoden 1/2

Wenn eine abgeleitete Klasse eine bereits in der Basisklasse definierte Methode **erneut definiert**, so „**überschreibt**“ diese **klassenspezifische Methode** die Methode der Basisklasse:

```
public class BasisKlasse {  
    void eineMethode() {  
        [...]  
    }  
}  
  
class AbgeleiteteKlasse extends BasisKlasse {  
    void eineMethode() {  
        super.eineMethode(); //ruft die Methode der Basisklasse auf  
        [...]  
    }  
}
```



Einige Möglichkeit
noch an die
Originalmethode
heranzukommen

Überschreiben von Methoden 2/2

Aus der abgeleiteten Klasse heraus kann die Methode der Basisklasse wieder über **super** erfolgen (siehe Beispiel vorherige Seite).

Aber **Objekte** vom Typ der **abgeleiteten Klasse** können nicht mehr die **ursprüngliche** Implementierung der Methode aufrufen.

Die spezielle Bedeutung der Überschreibung von Methoden erfolgt nächste Lehreinheit beim Thema Polymorphie.

```
1 public class VerdeckungUeberschreibung {
2     public static void main(String[] args) {
3         BasisKlasse b = new AbgeleiteteKlasse();
4
5         b.eineMethode();
6         ((BasisKlasse)b).eineMethode();
7
8         System.out.println(b.attribut);
9         System.out.println(((AbgeleiteteKlasse)b).attribut);
10    }
11 }
12
13 class BasisKlasse{
14     String attribut = "atrBasis";
15     void eineMethode() {
16         System.out.println("Basis");
17     }
18 }
19
20 class AbgeleiteteKlasse extends BasisKlasse{
21     String attribut = "atrAbgeleitet";
22     void eineMethode() {
23         System.out.println("Abgeleitet");
24         //super.eineMethode();
25     }
26 }
```

Abgeleitet
Abgeleitet
atrBasis
atrAbgeleitet

Aufgabe 8.3



Fügen Sie mit der Methode aus Aufgabe 8.1 (**addAttack()**) jedem Pokemon eine spezifische Attacke hinzu (siehe Tabelle).

Die neuen Attacken sollen im jeweiligen Konstruktor der spezifischen Pokemon hinzugefügt werden (*Glumanda*, *Shiggy*, *Bisasam* und *Bisaknosp*)

Pokemon Name	Attack-Name	Schaden/Typ
Glumanda	Glut	15/Feuer
Shiggy	Aquaknarre	15/Wasser
Bisasam	Rankenhieb	15/Pflanze
Bisaknosp	Rasierblatt	20/Pflanze

Überladen von Methoden

Wenn eine abgeleitete Klasse eine in der Basisklasse definierte **Methode mit einer anderen Parameterliste** (Signatur) erneut definiert, so wird die Methode der Basisklasse damit "**überladen**":

```
public class BasisKlasse {  
    void eineMethode() {  
        [...] }  
}  
  
class AbgeleiteteKlasse extends BasisKlasse {  
    void eineMethode(int parameter) {  
        [...]  
    }  
}
```

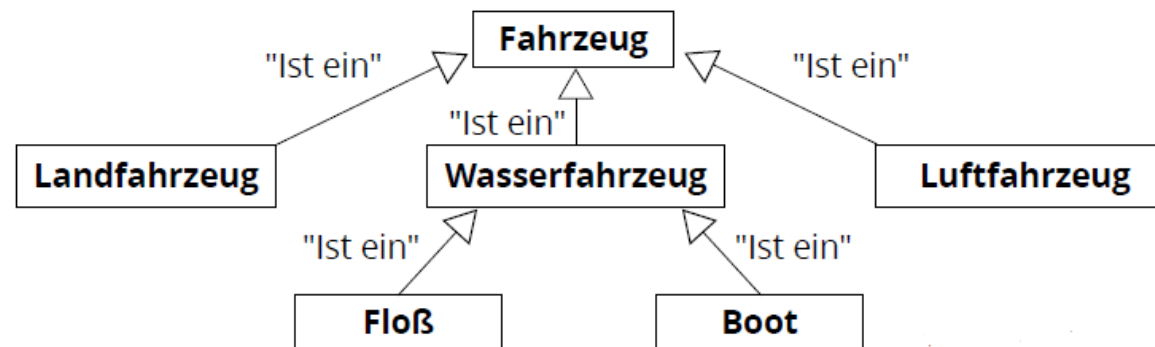
„Überladen“
der Methode
eineMethode()

Vgl. LE 06, Folie 18

Vererbung vs. Komposition 1/2

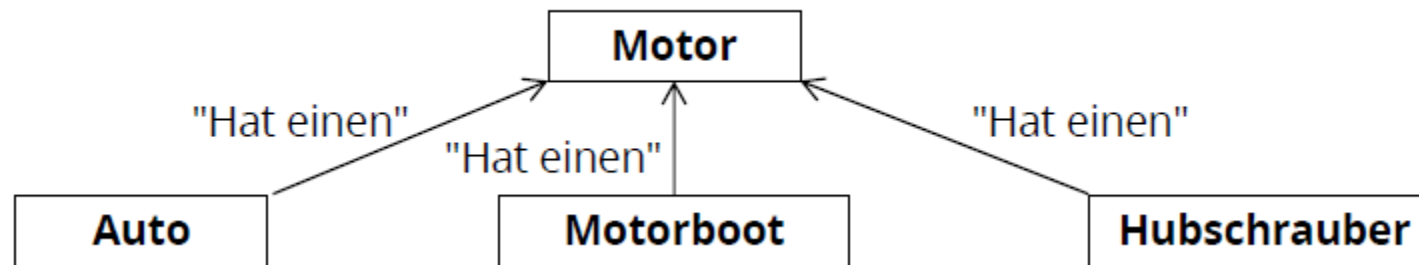
Sinnvolle Vererbung setzt voraus, dass...

- a)die abgeleitete Klasse eine Weiterführung oder Spezialisierung der Basisklasse ist oder...
- b)die Basisklasse eine allgemeinere oder abstraktere Beschreibung der Objekte der abgeleiteten Klasse darstellt ("Ist ein").



Vererbung vs. Komposition 2/2

Alternativ zum Erben von einer Basisklasse kann ein Objekt auch über eine **Assoziation** (vgl. LE 07, Folie 10) zu einer Eigenschaft gelangen. Man spricht dann auch von **Komposition** ("Hat ein"):



Die Komposition erlaubt es, einer Klasse **Eigenschaften** zu verleihen, die sie durch Vererbung allein **normalerweise nicht** bekommen würde (Beispiel: ein Boot mit Flugfähigkeiten).

Aufgabe 8.4



Prüfen Sie im bestehenden Projekt **alle Zugriffsmodifizierer** und ändern falls nötig.

Wo ein Zugriff von außen nicht nötig ist, soll dieser nicht gewährt werden.

Viele Dank für Ihre Aufmerksamkeit



Hausaufgabe



Standard:

Wir wollen unser **Schiggy** etwas stärker machen und überschreiben hierfür die Methode **takeDamage()**.

Die neue Methode soll immer einen Schaden weniger abziehen als bisher.

Bonus:

Versucht euch eine Mechanik zu überlegen, wie das Stein-Schere-Papier-Prinzip nach folgender Grafik umgesetzt werden kann.

- + doppelter Schaden
- halbiertes Schaden



		Verteidiger			
		NORMAL	FEUER	WASSER	PFLANZE
Angreifer	NORMAL				
	FEUER		-	-	+
	WASSER		+	-	-
	PFLANZE		-	+	-